

Module III

Classification is needed for both human and machine intelligence.

→ Deciding what letter, word or image has been presented to our senses, recognizing faces or voices, sorting mail, assigning grades to homeworks. These are all examples of assigning a category to an input.

→ Naive Bayes algorithm apply to text categorization, the task of assigning a label or category to an entire text or document.

→ One of the common text categorization task is sentiment analysis, the extraction of sentiment, the positive or negative orientation that a writer expresses towards some object.

→ A review of a movie, book or product on the web expresses the author's sentiment toward the product.

→ The simplest version of sentiment analysis is a binary classification task and the words of the review provide excellent cues.

Ex:- The following phrases extracted from +ve and -ve reviews of movies and restaurants.

words like great, richly, awesome, and pathetic, and awful, ridiculously are very informative cues.

→ Spam detection is the another important application of binary classification, task of assigning an email to one of the two classes spam or not-spam.

→ Many lexical and other features can be used to perform this classification.

Ex:- The ^{emails containing} phrases "Without any cost" or "Dear Winner" are suspicious.

→ Different languages will need different language processing techniques.

→ The task of language id is first step in the first step in most language processing pipelines.

→ Authorship Contribution is a text classification task, determining a text's author, are also relevant to digital humanities, social sciences & forensic linguistics.

→ The text classification task is assigning a library subject category or topic label to a text.

→ Classification is essential for these tasks

- * disambiguation
- * word tokenization
- * language modeling
- * part of speech tagging

→ The goal of classification is to take single observation, extract some useful features and thereby classify the observation into one of a set of discrete classes.

→ In Supervised learning, we have a dataset of input observations, each associated with some correct output.

The goal of algorithm is to learn how to map from a new observation to a correct output.

→ The task of Supervised classification is to take an input x and a fixed set of output classes $Y = \{y_1, y_2, \dots, y_n\}$ and return a predicted class $y \in Y$.

In Supervised ~~Situation~~ situation, we have a training set of N documents that have each been hand-labeled with a class: $\{(d_1, c_1), \dots, (d_N, c_N)\}$.

Our goal is to learn the classifier that is capable of mapping from a new document d to its correct class $c \in C$, where C is some set of useful document classes.

Naive classifiers :-

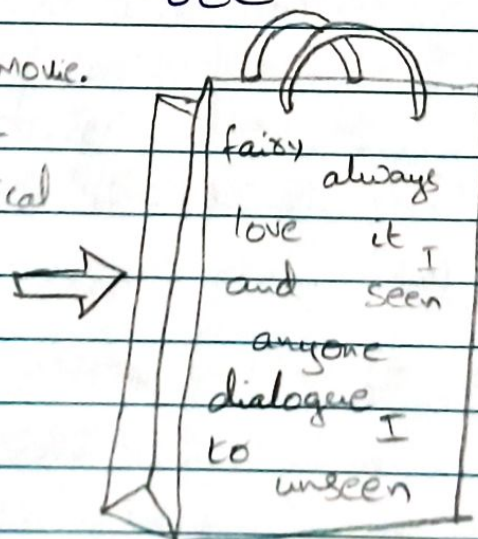
Multinomial naive Bayes classifier is a bayesian classifier that makes a simplifying (naive) assumption about how the features interact.

Ex:- we represent a text document as if it were a bag of words, that is an unordered set of words with their position ignored, keeping only their frequency in the document.

→ Naive Bayes is a probabilistic classifier meaning that for a document d , out of all classes $c \in C$ the classifier returns the class $\hat{c}$ which has the maximum posterior probability given the document.

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} p(c/d) \rightarrow \textcircled{1}$$

I love this movie.
It's sweet, but
with statistical
humour



it	6
I	5
the	4
to	3
and	3
Seen	3
yet	1
would	1
times	1
sweet	1

→ The Bayes rule is

$$p(x/y) = \frac{p(y/x) p(x)}{p(y)} \rightarrow (2)$$

by substituting (1) & (2) is

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} (p(c/d)) = \underset{c \in C}{\operatorname{argmax}} \frac{p(d/c) p(c)}{p(d)} \rightarrow (3)$$

$p(d)$ does not change for each class. Thus the simpler formula is

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} p(c/d) = \underset{c \in C}{\operatorname{argmax}} p(d/c) \cdot p(c) \rightarrow (4)$$

In above equation, it states a kind of implicit assumption about how a document is generated:

first a class is sampled from $p(c)$ and then the words are generated by sampling from $p(d/c)$.

→ This (4) equation called as generative model.

→ To do classification: we compute the most probable class $\hat{c}$ given some document d by choosing the class which has the highest probability product of two probabilities: the prior probability of the class $p(c)$ and the likelihood of the document $p(d/c)$:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \underbrace{p(d/c)}_{\text{likelihood}} \underbrace{p(c)}_{\text{prior}} \rightarrow (5)$$

→ With out loss of generality, we can represent a document d as a set of features $f_1, f_2, \dots, f_n$:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \underbrace{P(f_1, f_2, \dots, f_n | c)}_{\text{likelihood}} \cdot \underbrace{p(c)}_{\text{prior}} \rightarrow (6)$$

The equation (6) is too hard to compute directly. Therefore Naive Bayes classifiers make two assumptions:

1. The position doesn't matter
2. The probabilities $p(f_i | c)$ are independent given the class c .

Hence

$$p(f_1, f_2, \dots | c) = p(f_1 | c) \cdot p(f_2 | c) \cdot \dots \cdot p(f_n | c) \rightarrow (7)$$

So, the equation for Naive Bayes classifier is

$$C_{NB} = \underset{c \in C}{\operatorname{argmax}} p(c) \cdot \prod_{f \in F} p(f | c) \rightarrow (8)$$

→ To apply Naive Bayes classifier to text, we will use each word in the documents as a feature, and consider each of the words in the document by taking an index through every word position in the document

positions ← all word positions in text document

$$C_{NB} = \underset{c \in C}{\operatorname{argmax}} p(c) \prod_{i \in \text{positions}} p(w_i | c) \rightarrow (9)$$

→ Naive Bayes calculations, like calculations for language modeling, are done in log space to avoid underflow & increase speed.

Thus equation (9) becomes

$$C_{NB} = \underset{c \in C}{\operatorname{argmax}} \log P(c) + \sum_{i: \text{positions}} \log P(w_i/c) \quad \rightarrow (10)$$

→ Classifiers that use a linear combination of the inputs to make classification decision like Naive Bayes (equation 10) and logistic regression are called "linear classifiers"

Training the Naive Bayes classifier:-

Let

N_c ^{be} = no. of documents in our training data with class c

N_{doc} - The total no. of documents.

Then
$$\hat{P}(c) = \frac{N_c}{N_{\text{doc}}}$$

And we will compute the fraction of times the word w_i appears among all words in all documents of topic c & all documents with category c into one big "category c " text. Then

$$\hat{P}(w_i/c) = \frac{\text{Count}(w_i, c)}{\sum_{w \in V} \text{Count}(w, c)}$$

Here V is vocabulary consists of the union of all words in all classes.

$$E_i: p(\text{'fantastic'}/\text{positive}) = \frac{\text{Count}(\text{'fantastic'}, \text{positive})}{\sum_{w \in V} \text{Count}(w, \text{positive})}$$

There is a problem with this estimation.

If no training documents containing the word fantastic are classified as positive. In such cases the probability will be zero.

The simplest solution is the add-one smoothing. So

$$p(w_i | c) = \frac{\text{Count}(w_i, c) + 1}{\sum_{w \in V} (\text{Count}(w, c) + 1)} = \frac{\text{Count}(w_i, c) + 1}{\sum_{w \in V} \text{Count}(w, c) + |V|}$$

→ The words which did not occur in any training document in any class are considered as unknown words.

The solution for such unknown words is to ignore them - remove them from the text document.

→ Stop words - very frequent words like a, the need to be ignored.

→ The below figure shows the Naive Bayes algorithm using add-1 smoothing

4.2. Naive Bayes algorithm

→ function TRAIN NAIVEBAYES (D, C) returns $V \log p(c) \log p(w|c)$

for each class $c \in C$ # Calculate $p(c)$ terms

N_{doc} = no. of documents in D

N_c = no. of documents from D in class c

$\logprior[c] \leftarrow \log \frac{N_c}{N_{doc}}$

$V \leftarrow$ vocabulary of D

$bigdoc[c] \leftarrow$ append (d) for $d \in D$ with class c

for each word w in V # Calculate $p(w|c)$ terms

$Count(w, c) \leftarrow$ # of occurrences of w in $bigdoc[c]$

$\loglikelihood[w, c] \leftarrow \log \frac{Count(w, c) + 1}{\sum_{w' \in V} (Count(w', c) + 1)}$

return $\logprior, \loglikelihood, V$

→ function ~~TRAIN~~ TEST NAIVEBAYES ($testdoc, \logprior, \loglikelihood, C, V$) returns best c

for each class $c \in C$

$sum[c] \leftarrow \logprior[c]$

for each position i in $testdoc$

word $\leftarrow testdoc[i]$

if word $\in V$

$sum[c] \leftarrow sum[c] + \loglikelihood[word, c]$

return $\argmax_c sum[c]$

worked example:-

(Q) We will use sentiment analysis domain with 2 classes +ve & -ve. and take the following training and test documents simplified from actual movie reviews.

cat	Document
Training -	just plain boring
-	entirely predictable and lacks energy
-	no surprises and very few laughs
+	very powerful
+	the most fun film of the summer
Test	? predictable with no fun

The prior $p(c)$ for 2 classes is computed using $\frac{N_c}{N_{doc}}$

$$p(-) = \frac{3}{5} \quad p(+) = \frac{2}{5}$$

→ The word with doesn't occur in training set so we drop it completely (we don't use unknown words for Naive Bayes).

Hence the likelihoods from the training set for the remaining 3 words "predictable"

"no"

"fun"

$$p(\text{"predictable"}/-) = \frac{1+1}{14+20}$$

$$p(\text{"predictable"}/+) = \frac{0+1}{9+20}$$

$$p(\text{"no"}/-) = \frac{1+1}{14+20}$$

$$p(\text{"no"}/+) = \frac{0+1}{9+20}$$

$$p(\text{"fun"}/-) = \frac{0+1}{14+20}$$

$$p(\text{"fun"}/+) = \frac{1+1}{9+20}$$

for test sentence $S = \text{"predictable with no fun"}$

after removing the word with, the class is

$$p(-) \cdot p(S/-) = p(-) \cdot p(\text{"predictable"}/-) \cdot p(\text{"no"}/-) \cdot p(\text{"fun"}/-)$$

$$= \frac{3}{5} \times \frac{2}{34} \times \frac{2}{34} \times \frac{1}{34} = 6.1 \times 10^{-5}$$

$$p(+). p(S/+) = p(+). p(\text{"predictable"}/+) \cdot p(\text{"no"}/+) \cdot p(\text{"fun"}/+)$$

$$= \frac{2}{5} \cdot \frac{1}{29} \cdot \frac{1}{29} \cdot \frac{2}{29} = 3.2 \times 10^{-5}$$

Thus the model predicts the class negative for the test sentence.

Optimizing for Sentiment Analysis:-

Naive Bayes text classification can work well for sentiment analysis, some small changes are (needed) employed that improve performance

① For sentiment classification and a no. of other classification tasks, whether a word occurs or not, doesn't considering the frequency.

This variant is called "binary multinomial naive Bayes" or "binary naive Bayes"

→ This variant uses the same algorithm (refer back 4.2) excepts that ^{for} each document we remove all duplicate words before concatenating them in to the single big document during training and we also remove duplicate words from test documents, as shown below.

Four original documents	NB Counts		Binary Counts	
	+	-	+	-
• it was pathetic the worst				
was the boxing scenes	and	2	0	1
- no plot twists or great	boxing	0	1	0
scenes	film	1	0	1
+ and satire and great plot	great	3	1	2
twists	it	0	1	0
+ great scene great film	no	0	1	0
	or	0	1	0
→ After per document binarization:	past.	0	1	0
- It was pathetic the	pathetic	0	1	0
worst past boxing scenes	plot	1	1	1

Page No.:

		+	-	+	-
- no plot twists or	Satire	1	0	1	0
great scenes	scenes	1	2	1	2
+ and satire great	the	0	2	0	1
plot twists	twists	1	1	1	1
+ great scenes film	was	0	2	0	1
	worst	0	1	0	1

② A second important addition of text classification for sentiment is to deal with negation.

During text normalization, prepend the prefix NOT to every word after a token of logical negation (n't, not, no, never) until the next punctuation mark.

Thus the phrase,
did n't like this movie; but I
becomes

didn't NOT-like NOT-this NOT-movie, but I

Newly formed words like NOT-like, NOT-recommend will thus occur more often in negative documents and act as cues for negative sentiment, while words like NOT-bored, NOT-dismiss will acquire positive associations.

Syntactic parsing can be used for more accurately with the scope relationship between these negation words and the predicates they modify.

→ Sentiment lexicons, list of words that are pre-annotated with positive or negative sentiment.

→ Four popular lexicons are

- * General Inquirer
- * LIWC
- * Opinion lexicon
- * MPQA

→ Subjective lexicon MPQA has 6885 words each marked for +ve or -ve.

Ex:- +: admirable, beautiful, favor, glee, great
-: bad, bias, cheat, foul, harsh, hate

→ Naive Bayes for other text classification tasks

The task of spam detection is deciding a particular piece of email is an example of spam.

→ It is one of the first applications of naive Bayes to text classification.

→ It will use a set of words or phrases as features

Ex:- one hundred percent guaranteed millions of dollars. If mail has these kind of words then it will be identified as spam.

→ It also includes features like HTML has a low ratio of text to image area.

→ More Sample Spam features:

- * Email subject line is all capital letters
- * Contains phrases of urgency like "urgent reply"
- * Email subject line contains "online pharmaceutical"
- * HTML has unbalanced "head" tags
- * Claims you can be removed from the list

→ For other task, like language id- determining what language a given piece of text is written- in the most effective naive Bayes features are not words at all, but character n-grams, like

2-grams ('zw')

3-grams ('nyd') ('vow')

4-gram ('zeai') ('thep') or

even byte n-gram, where instead of using the multibyte Unicode character representations called "codepoints"

→ A widely used naive Bayes System, langid.py begins with all possible n-grams of length 1-4, using feature selection to winnow down to the most informative 7000 final features.

→ Language ID systems are trained on multilingual text such as Wikipedia text in 68 different languages was used in.

→ Naive Bayes as a Language Model :-

The likelihood features from the naive Bayes model assign a probability to each word $p(\text{word}/c)$ the model also assign a probability to each sentence.

$$p(s/c) = \prod_{i \text{ positions}} p(w_i/c)$$

Thus Consider a naive Bayes model with the classes positive (+) and negative (-) and the following model parameters:

<u>w</u>	<u>$p(w/+)$</u>	<u>$p(w/-)$</u>
I	0.1	0.2
love	0.1	0.001
this	0.01	0.01
fun	0.05	0.005
film	0.1	0.1

Each of the two columns above instantiates a language model that can assign a probability to the sentence "I love this fun film"

$$p(\text{I love this fun film}/+) = 0.1 \times 0.1 \times 0.01 \times 0.05 \times 0.1 = 5 \times 10^{-7}$$

$$p(\text{I love this fun film}/-) = 0.2 \times 0.001 \times 0.01 \times 0.005 \times 0.1 = 1.0 \times 10^{-9}$$

$p(s/\text{pos}) > p(s/\text{neg})$. So the naive Bayes classifier assigns positive class to this sentence.

Evaluation: precision, Recall F-measure:-

The human defined labels for each document are referred to as "gold labels"

→ To evaluate any system for detecting things, we start by building a confusion matrix.

Confusion matrix is a table for visualizing how an algorithm performs with respect to the human gold labels using two dimensions (system output and gold labels) and each cell labeling a set of possible outcomes.

fig. Confusion matrix
Gold Standard labels

		gold positive	gold negative	
System output labels	System positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	System negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+fn+tn}$

→ precision measures the percentage of items that the system detected that are in fact positive.

precision is defined as

$$\text{precision} = \frac{\text{true positive}}{\text{true positives} + \text{false positive}}$$

→

→ Recall measures the percentage of items actually present in the input that were correctly identified by the system. Recall is defined as

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

→ The combination of precision and recall is the F-measure, defined as

$$F_{\beta} = \frac{(\beta^2 + 1) P R}{\beta^2 P + R}$$

The β parameter differentially weights the importance of recall and precision based on needs of an application.

Values of $\beta > 1$ favor recall

$\beta < 1$ favor precision

$\beta = 1$ then precision & recall are

equally balanced.

→ Then F-measure $F_{\beta=1}$ or $F_1 = \frac{2PR}{P+R}$

→ The F-measure comes from a weighted harmonic mean of precision and recall.

The harmonic mean of a set of numbers is the reciprocal of the arithmetic mean of reciprocals.

$$\text{Harmonic Mean}(a_1, a_2, \dots, a_n) = \frac{n}{\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \dots + \frac{1}{a_n}}$$

Hence F-measure is

$$F = \frac{1}{\alpha \frac{1}{P} + (1-\alpha) \frac{1}{R}}$$

$$(or) \left(\text{with } \beta^2 = \frac{1-\alpha}{\alpha} \right) F = \frac{(\beta^2 + 1) PR}{\beta^2 P + R}$$

⇒ Evaluating with more than two classes :-

→ The classification tasks in language processing have more than two classes.

The following figure shows a Confusion matrix for a three class categorization task

		gold labels			
		urgent	normal	spam	
System output	urgent	8	10	1	precision _u = $\frac{8}{8+10+1}$
	normal	5	60	50	precision _n = $\frac{60}{5+60+50}$
	spam	3	30	200	precision _s = $\frac{200}{3+30+200}$
		recall _u	recall _n	recall _s	
		= $\frac{8}{8+5+3}$	= $\frac{60}{10+60+30}$	= $\frac{200}{1+50+200}$	

→ In macro averaging, we compute the performance for each class and then average over classes.

→ In Micro averaging, we collect the decisions for all classes in to a single confusion matrix and then compute precision and recall

class 1: Urgent

	true Urgent	false not
System Urgent	8	11
System not	8	340

	true Urgent	false not
System Urgent	8	11
System not	8	340

$$\text{precision} = \frac{8}{8+11} = 0.42$$

class 2: Normal

	true Normal	false Not
System Normal	60	55
System not	40	212

	true Normal	false Not
System Normal	60	55
System not	40	212

$$\text{precision} = \frac{60}{60+55} = 0.52$$

class 3: Spam

	true Spam	false not
System Spam	200	33
System not	51	83

	true Spam	false not
System Spam	200	33
System not	51	83

$$\text{precision} = \frac{200}{200+33} = 0.86$$

pooled

	true Yes	false No
System Yes	268	99
System No	99	635

	true Yes	false No
System Yes	268	99
System No	99	635

micro average

$$= \frac{268}{268+99} = 0.73$$

$$\text{macro average} = \frac{0.42 + 0.52 + 0.86}{3}$$

$$= 0.60$$

Test sets and cross validation:-

We use the training set to train the model, then use the development test set (also called devset) to tune some parameters to decide what the best model is.

→ The test set may not be large enough to be representative. The solution for this is "cross validation".

→ In cross validation, we choose a number k , and partition our data into k disjoint subsets called "folds".

Now we choose one of those k folds as a test set, train our classifier on the remaining $k-1$ folds and then compute the error rate on the test set.

Then we repeat with another fold as the test set, again training on the other $k-1$ folds.

Training iterations

1	Dev	Training	
2		Dev	Training
3			Dev Training
4			Dev Training
⋮			⋮
9	Training		Dev
10			Dev

Testing

Test
set

If $k=10$, then it is called 10-fold cross validation.

→ The problem with this cross validation is that because all the data is used for testing.

→ Statistical Significance Testing:—

Suppose we are comparing the performance of classifiers A and B on a metric M such as F_1 or accuracy.

→ Let $M(A, x)$ the score that system A gets on test set x and $S(x)$ the performance difference between A & B on x :

$$S(x) = M(A, x) - M(B, x)$$

$S(x)$ is called the effect size, a bigger S means that A seems to be way better than B, a small S means A seems to be only a little better.

→ In statistical hypothesis testing, A's superiority over B can be tested by formalizing two hypotheses

$$H_0 : S(x) \leq 0$$

$$H_1 : S(x) > 0$$

The hypothesis H_0 called the null hypothesis, suppose that $S(x)$ is negative or zero that A is not better than B.

→ we formalize the likelihood as p-value: the probability assuming the null hypothesis H_0 is true, of seeing the $S(x)$ that we saw or one even greater.

$$P(S(x) \geq S(u) | H_0 \text{ is true})$$

→ If "A is better than B" is statistically significant if the S has a probability that is below the threshold and we therefore reject the null hypothesis.

→ There are two non-parametric tests used in NLP:

- ① approximate randomization
- ② bootstrap test

→ Here we will describe bootstrap test (paired version)

→ paired tests are those in which we compare two sets of observations that are aligned: Each observation in one set can be paired with an observation in another.

→ Paired Bootstrap Test :-

→ The bootstrap test can apply to any metric: precision, recall or F1.

→ The word bootstrapping refers to repeatedly drawing large no. of samples with replacement (called bootstrap samples) from an original set.

Consider a tiny classification example with a set x of 10 documents. The first row of below figure shows the results of two classifiers A & B on this test set.

Each document is labeled x by one of four possibilities

(i) A & B both right - AB

(ii) A & B both wrong - AB

(iii) A right & B wrong - AB

(iv) A wrong & B right - AB

→ We assume metric is accuracy, A has accuracy of 0.70 & B has 0.50 so $S(x)$ is 0.20

	1	2	3	4	5	6	7	8	9	10	A%	B%	$S(x)$
x	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	0.70	0.50	0.20
$x^{(1)}$	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	0.60	0.60	0.00
$x^{(2)}$	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	0.60	0.70	0.10
$\vdots$													
$x^{(b)}$													

Now we create a large number b of virtual test sets $x^{(i)}$, each of size $n=10$.

To create each virtual test $x^{(i)}$ we repeatedly select a cell from row x with replacement.
 Ex:- to create the first cell of the first virtual test set $x^{(1)}$, we randomly select the second cell of the x row, we copy the value AB in to our new cell, and move on to create the second cell of $x^{(1)}$, each time sampling from the original x with replacement.

Now
$$p\text{-value}(x) = \frac{1}{b} \sum_{i=1}^b \mathbb{1}(S(x^i) - S(x) \geq 0)$$

$\mathbb{1}(x)$ mean 1 if x is true & 0 otherwise

If $S(x^i)$ exceeds the expected value of $S(x)$ by $S(x)$ or more.

$$p\text{-value}(x) = \frac{1}{b} \sum_{i=1}^b \mathbb{1}(S(x^i) - S(x) \geq S(x))$$

$$= \frac{1}{b} \sum_{i=1}^b \mathbb{1}(S(x^i) \geq 2S(x))$$

function BOOTSTRAP(test set x , no. of samples b) returns $p\text{-value}(x)$

Calculate $S(x)$ # how much better does algorithm A do than B on x

$S = 0$

for $i = 1$ to b do

for $j = 1$ to n do

select a member of x at random & add it to x^i

calculate $S(x^i)$

$S \leftarrow S + 1$ if $S(x^i) \geq 2S(x)$

$$p\text{-value}(x) \approx \frac{S}{b}$$

return $p\text{-value}(x)$

Avoiding Harms in Classification:-

It is important to avoid harms that may result from classifiers (like naive Bayes classifier & other classification algorithms)

→ One class of harms is: representational harms harms caused by a system that demeans a social group for example by perpetuating negative stereotypes about them

→ In some classifiers, tasks may lead to both representational harms and other harms such as silencing

Ex:- The imp text classification task is toxicity detection

Toxicity detection is the task of detecting hate speech, abuse, harassment, or other kinds of toxic language.

While the goal of such classifiers is to help reduce societal harm, toxicity classifiers can themselves cause harms. These model problems can be caused by biases or other problems in training data.

→ One way to study these kinds of factors and make them clear is releasing a "model card" for each version of a model.

Model card documents on a ML model with info like :

- * training alg & parameters

- * training data sources & preprocessing

- * Evaluation data sources

- * model performance across different

- * intended use & users.

Situations